Dokumentacja implementacyjna

System wizualizacji grafów planarnych

Projekt w języku C

Autorzy:
Krzysztof Wasilewski, Jakub Pietrzkiewicz

Data: 24.03.2026

Spis treści

1 Wstęp ... 3
1.1 Cel dokumentu ... 3
1.2 Zakres ... 3
2 Architektura systemu ... 4
2.1 Podział na moduły ... 4
2.2 Przepływ wykonania ... 4
3 Model danych ... 6
3.1 Struktury podstawowe grafu ... 6
3.2 Struktury pomocnicze ... 6
4 Interfejsy modułów ... 7
4.1 Interfejs funkcji modułu graph ... 7
4.2 Interfejs funkcji modułu fruchterman ... 7
4.3 Interfejs funkcji modułu tutte ... 7
4.4 Interfejs funkcji modułu adjacency_list ... 7
4.5 Interfejs funkcji modułu utils ... 7
5 Specyfikacja danych wejściowych i wyjściowych ... 8
5.1 Format wejściowy ... 8
5.2 Format wyjściowy tekstowy ... 8
5.3 Format wyjściowy binarny ... 8
6 Implementacja algorytmów ... 10
6.1 Fruchterman-Reingold ... 10
6.2 Tutte ... 10
7 Zarządzanie pamięcią ... 13
7.1 Zwalnianie ... 13
8 Obsługa błędów ... 14
8.1 Komunikaty i kody powrotu ... 14
9 Kompilacja i testowanie ... 15
9.1 Kompilacja ... 15
9.2 Testy ... 15

1 Wstęp

1.1 Cel dokumentu

Celem dokumentacji implementacyjnej jest przedstawienie szczegółów budowy aplikacji konsolowej w języku C do wyznaczania współrzędnych węzłów grafu planarnego. Dokument opisuje rzeczywistą strukturę kodu, przepływ sterowania, formaty danych wejścia i wyjścia oraz obsługę błędów.

1.2 Zakres

Dokument obejmuje:

- architekturę modułową programu,
- model danych i struktury pamięci,
- implementację algorytmów Fruchterman-Reingold i Tutte,
- specyfikację interfejsów wejścia/wyjścia,
- kody powrotu i scenariusze błędowe,
- sposób budowania i uruchamiania testów.

2 Architektura systemu

2.1 Podział na moduły

Implementacja projektu została podzielona na następujące moduły:

- `main` - parsowanie argumentów CLI i orkiestracja całego przebiegu programu,
- `graph` - odczyt i zapis grafu oraz mapowanie identyfikatorów węzłów,
- `fruchterman` - obliczanie układu metodą siłową,
- `tutte` - obliczanie osadzenia w oparciu o iteracyjną relaksację,
- `adjacency_list` - konwersja grafu do list sąsiedztwa dla algorytmu Tutte,
- `utils` - pomocnicza lista jednokierunkowa i wspólne kody statusu.

Każdy moduł ma dedykowany plik nagłówkowy (`.h`) zawierający publiczny interfejs funkcji oraz plik implementacyjny (`.c`) z logiką wewnętrzną.

2.2 Przepływ wykonania

Podstawowy przebieg programu:

1. Parsowanie argumentów i ustawienie wartości domyślnych.
2. Walidacja parametrów (`iterations > 0`, `temperature > 0`, `size > 0`).
3. Wczytanie grafu z pliku tekstowego (`load_graph`).
4. Wykonanie wybranego algorytmu (`run_fruchterman` lub `run_tutte`).
5. Zapis wyników (`save_graph_as_text` albo `save_graph_as_binary`).
6. Zwolnienie pamięci (`free_graph`) i zwrócenie kodu statusu.

Wszystkie błędy propagowane są przez kody zwracane funkcji.

Fragment implementacji walidacji argumentów wejściowych:

```
if (input_file == NULL || output_file == NULL) {
  fprintf(stderr, "Error: Input and output files must be specified.\n");
  return ARGUMENTS_ERROR;
}
if (iterations <= 0) {
  fprintf(stderr, "Error: Invalid iteration value\n");
  return ARGUMENTS_ERROR;
}
if (temperature <= 0) {
  fprintf(stderr, "Error: Invalid temperature value\n");
  return ARGUMENTS_ERROR;
}
if (size <= 0) {
  fprintf(stderr, "Error: Invalid size value\n");
  return ARGUMENTS_ERROR;
}
```

Fragment implementacji odpowiedzialny za wybór algorytmu i formatu zapisu:

```
if (strcmp(algorithm, "fruchterman") == 0) {
  status = run_fruchterman(graph, temperature, iterations, size);
} else if (strcmp(algorithm, "tutte") == 0) {
  status = run_tutte(graph, size, iterations);
} else {
  fprintf(stderr,
          "Error: Unknown algorithm '%s'. Use 'fruchterman' or 'tutte'.\n",
```

```
            algorithm);
    free_graph(graph);
    return ARGUMENTS_ERROR;
  }

  if (strcmp(format, "text") == 0) {
    status = save_graph_as_text(graph, output_file);
  } else if (strcmp(format, "binary") == 0) {
    status = save_graph_as_binary(graph, output_file);
  } else {
    fprintf(stderr, "Error: Unknown format '%s'. Use 'text' or 'binary'.\n",
            format);
    free_graph(graph);
    return ARGUMENTS_ERROR;
  }
```

3 Model danych

3.1 Struktury podstawowe grafu

W module `graph` używane są trzy główne struktury:

- `Node`
  - `id` - identyfikator logiczny węzła,
  - `x`, `y` - współrzędne geometryczne.
- `Edge`
  - `first_node_index`, `second_node_index` - indeksy w tablicy `nodes`,
  - `weight` - waga krawędzi,
  - `label[33]` - etykieta krawędzi, maksymalnie 32 znaki + `\0`.
- `Graph`
  - `nodes` - dynamiczna tablica węzłów,
  - `nodes_count` - liczba węzłów,
  - `edges` - dynamiczna tablica krawędzi,
  - `edges_count` - liczba krawędzi.

3.2 Struktury pomocnicze

Dodatkowe struktury wykorzystywane przez algorytmy:

- `List` (moduł `utils`) - jednokierunkowa lista do budowy zbioru unikalnych ID przy odczycie grafu,
- `Neighbor` (moduł `adjacency_list`) - pojedynczy element listy sąsiadów,
- `AdjacencyList` - tablica list sąsiadów i tablica stopni węzłów.

Fragment implementacji budowy listy sąsiedztwa:

```
for (int i = 0; i < graph->edges_count; i++) {
  int first_index = graph->edges[i].first_node_index;
  int second_index = graph->edges[i].second_node_index;
  double weight = graph->edges[i].weight;

  if (add_neighbor(&adj_list->adjacency_list[first_index], second_index,
                   weight) != SUCCESS ||
      add_neighbor(&adj_list->adjacency_list[second_index], first_index,
                   weight) != SUCCESS) {
    free_adjacency_list(adj_list);
    return MEMORY_ERROR;
  }
  adj_list->degrees[first_index]++;
  adj_list->degrees[second_index]++;
}
```

4 Interfejsy modułów

4.1 Interfejs funkcji modułu graph

Funkcje publiczne:

- `int load_graph(char path[], Graph **graph_out)`
- `int save_graph_as_binary(Graph *graph, char path[])`
- `int save_graph_as_text(Graph *graph, char path[])`
- `int free_graph(Graph *graph)`
- `int get_node_index(Graph *graph, int node_id)`

Wczytywanie grafu realizowane jest w dwóch przebiegach przez plik:

- przebieg 1: zliczenie krawędzi i identyfikacja unikalnych wierzchołków,
- przebieg 2: właściwe mapowanie krawędzi na indeksy tablicowe.

Węzły są sortowane po `id` i wyszukiwane binarnie (`bsearch`), co przyspiesza mapowanie identyfikatorów.

Fragment implementacji mapowania identyfikatora węzła na indeks tablicy:

```
int get_node_index(Graph *graph, int node_id) {
  Node key = {.id = node_id};
  Node *result = (Node *)bsearch(&key, graph->nodes, graph->nodes_count,
                                 sizeof(Node), compare_nodes_by_id);
  return (result == NULL) ? -1 : (int)(result - graph->nodes);
}
```

4.2 Interfejs funkcji modułu fruchterman

- `int run_fruchterman(Graph *graph, double initial_temperature, int max_iterations, double size)`

4.3 Interfejs funkcji modułu tutte

- `int run_tutte(Graph *graph, double size, int max_iterations)`

4.4 Interfejs funkcji modułu adjacency_list

- `int create_adjacency_list(Graph *graph, AdjacencyList **adj_list_out)`
- `void free_adjacency_list(AdjacencyList *adj_list)`

4.5 Interfejs funkcji modułu utils

- `int list_contains(List *head, int value)`
- `int list_prepend(List **head, int value)`
- `void list_free(List *head)`

Moduł zawiera również wspólne kody statusu:

- `SUCCESS = 0`
- `ARGUMENTS_ERROR = 1`
- `FILE_ERROR = 2`
- `INPUT_FORMAT_ERROR = 3`
- `ALGORITHM_ERROR = 4`
- `MEMORY_ERROR = 5`

5 Specyfikacja danych wejściowych i wyjściowych

5.1 Format wejściowy

Program przyjmuje tekstowy plik wejściowy o rekordach:

```
<label> <nodeA> <nodeB> <weight>
```

Szczegóły parsowania:

- `label` jest wczytywany przez `%32s` (maksymalnie 32 znaki),
- `nodeA`, `nodeB` są typu `int`,
- `weight` jest typu `double`,
- linie komentarza rozpoczynające się od # są pomijane,
- komentarze inline po danych również są poprawnie pomijane,
- białe znaki (`space`, `tab`, `CR`, `LF`) są tolerowane.

Fragment implementacji odczytu linii wejściowej:

```
int result = fscanf(file, "%32s %d %d %lf", label, &first_node,
                    &second_node, &weight);

if (result == 4) {
  edges_count++;
  int ids[2] = {first_node, second_node};
  for (int i = 0; i < 2; i++) {
    if (!list_contains(unique_nodes, ids[i])) {
      list_prepend(&unique_nodes, ids[i]);
      nodes_count++;
    }
  }
} else if (result != EOF) {
  return INPUT_FORMAT_ERROR;
}
```

5.2 Format wyjściowy tekstowy

Każdy rekord opisuje pojedynczy węzeł:

```
<node_id> <x> <y>
```

Fragment implementacji zapisu tekstowego:

```
for (int i = 0; i < graph->nodes_count; i++) {
  fprintf(file, "%d %lf %lf\n", graph->nodes[i].id,
          graph->nodes[i].x, graph->nodes[i].y);
}
```

5.3 Format wyjściowy binarny

Plik binarny to ciąg rekordów stałej długości 20 bajtów:

- 4 bajty: `int id`,
- 8 bajtów: `double x`,
- 8 bajtów: `double y`.

Fragment implementacji zapisu binarnego:

```c
for (int i = 0; i < graph->nodes_count; i++) {
  fwrite(&graph->nodes[i].id, sizeof(int), 1, file);
  fwrite(&graph->nodes[i].x, sizeof(double), 1, file);
  fwrite(&graph->nodes[i].y, sizeof(double), 1, file);
}
```

6 Implementacja algorytmów

6.1 Fruchterman-Reingold

Algorytm używa modelu siłowego:

- siła odpychania między każdą parą węzłów,
- siła przyciągania dla węzłów połączonych krawędzią,
- ograniczenie pozycji do kwadratu o boku `size`.

Wzory użyte w kodzie:

- $k = \sqrt{\frac{\text{area}}{|V|}}$ gdzie $\text{area} = \text{size} \cdot \text{size}$
- $f_r = \frac{k^2}{d}$
- $f_a = w \cdot \frac{d^2}{k}$

Fragment implementacji obliczania sił odpychania:

```
for (int a = 0; a < liczba_w; a++) {
  for (int b = 0; b < liczba_w; b++) {
    if (a != b) {
      double dx = graph->nodes[a].x - graph->nodes[b].x;
      double dy = graph->nodes[a].y - graph->nodes[b].y;
      double odleglosc = sqrt(dx * dx + dy * dy) + 1e-9;
      double fr = (k * k) / odleglosc;
      sila_x[a] += (dx / odleglosc) * fr;
      sila_y[a] += (dy / odleglosc) * fr;
    }
  }
}
```

Fragment implementacji aktualizacji pozycji i ograniczenia do obszaru:

```
for (int i = 0; i < liczba_w; i++) {
  double wek_sily = sqrt(sila_x[i] * sila_x[i] + sila_y[i] * sila_y[i]);
  if (wek_sily > 0) {
    double przesuniecie = fmin(wek_sily, temperature);
    graph->nodes[i].x += (sila_x[i] / wek_sily) * przesuniecie;
    graph->nodes[i].y += (sila_y[i] / wek_sily) * przesuniecie;
  }

  if (graph->nodes[i].x < 0.0)
    graph->nodes[i].x = 0.0;
  if (graph->nodes[i].x > size)
    graph->nodes[i].x = size;
  if (graph->nodes[i].y < 0.0)
    graph->nodes[i].y = 0.0;
  if (graph->nodes[i].y > size)
    graph->nodes[i].y = size;
}
```

6.2 Tutte

Implementacja Tutte przebiega etapami:

1. Dla grafów z mniej niż 4 węzłami stosowane są pozycje specjalne.
2. Dla większych grafów tworzona jest lista sąsiedztwa.
3. Wyznaczany jest cykl brzegowy.

4. Węzły cyklu rozmieszczane są równomiernie na obwodzie kwadratu.
5. Węzły wewnętrzne inicjalizowane są w środku obszaru.
6. Iteracyjna relaksacja przesuwa każdy węzeł wewnętrzny do ważonego środka sąsiadów.

W praktyce wyznaczenie ramy odbywa się dwuetapowo:

- najpierw funkcja `find_cycle_perimeter` próbuje znaleźć cykl brzegowy heurystycznie,
- jeżeli to się nie uda, `find_cycle_backup` wybiera 4 węzły o najwyższych stopniach jako ramę awaryjną.

Heurystyka startuje od węzła o najmniejszym stopniu i buduje ścieżkę po sąsiadach. Preferowany jest kandydat z mniejszą liczbą wspólnych sąsiadów z węzłem bieżącym (a przy remisie z mniejszym stopniem). Po domknięciu ścieżki do węzła startowego otrzymujemy cykl, który jest rozmieszczany na obwodzie kwadratu.

Fragment implementacji przełączania między wariantem podstawowym i awaryjnym:

```
static int find_cycle(AdjacencyList *adj_list, int **cycle, int *cycle_len) {
  return (find_cycle_perimeter(adj_list, cycle, cycle_len) == SUCCESS)
             ? SUCCESS
             : find_cycle_backup(adj_list, cycle, cycle_len);
}
```

Fragment implementacji pojedynczego kroku relaksacji:

```
Neighbor *neighbor = adj_list->adjacency_list[i];
while (neighbor != NULL) {
  int neighbor_idx = neighbor->node_index;
  double weight = neighbor->weight;

  sum_weighted_x += weight * graph->nodes[neighbor_idx].x;
  sum_weighted_y += weight * graph->nodes[neighbor_idx].y;
  sum_weights += weight;

  neighbor = neighbor->next;
}

if (sum_weights > 0.0) {
  graph->nodes[i].x = sum_weighted_x / sum_weights;
  graph->nodes[i].y = sum_weighted_y / sum_weights;
}
```

Fragment implementacji rozmieszczenia cyklu na obwodzie kwadratu:

```
double perimeter = 4.0 * size;
double step = perimeter / cycle_len;
for (int i = 0; i < cycle_len; i++) {
  double pos = i * step;
  double x, y;

  if (pos < size) {
    x = pos;
    y = 0.0;
  } else if (pos < 2.0 * size) {
    x = size;
    y = pos - size;
  } else if (pos < 3.0 * size) {
    x = size - (pos - 2.0 * size);
    y = size;
```

```
  } else {
    x = 0.0;
    y = size - (pos - 3.0 * size);
  }
  graph->nodes[cycle[i]].x = x;
  graph->nodes[cycle[i]].y = y;
}
```

7 Zarządzanie pamięcią

7.1 Zwalnianie

Pamięć jest zwalniana warstwowo: najpierw struktury zagnieżdżone (listy sąsiadów), potem tablice i na końcu obiekt nadrzędny. To ogranicza ryzyko wycieków i podwójnego zwalniania.

W przypadku błędów alokacji program od razu przerywa dalsze kroki i wykonuje cleanup tylko dla zasobów, które zostały już poprawnie utworzone.

Fragment implementacji zwalniania struktury grafu:

```
int free_graph(Graph *graph) {
  if (graph == NULL)
    return SUCCESS;
  if (graph->nodes)
    free(graph->nodes);
  if (graph->edges)
    free(graph->edges);
  free(graph);
  return SUCCESS;
}
```

Fragment implementacji zwalniania listy sąsiedztwa:

```
for (int i = 0; i < adj_list->nodes_count; i++) {
  Neighbor *current = adj_list->adjacency_list[i];
  while (current != NULL) {
    Neighbor *next = current->next;
    free(current);
    current = next;
  }
}
free(adj_list->adjacency_list);
free(adj_list->degrees);
free(adj_list);
```

8 Obsługa błędów

8.1 Komunikaty i kody powrotu

Program raportuje błędy na `stderr` i kończy działanie kodem:

- `0` - sukces,
- `1` - błąd argumentów/parametrów,
- `2` - błąd operacji plikowych,
- `3` - błąd formatu danych wejściowych,
- `4` - błąd algorytmu,
- `5` - błąd alokacji pamięci.

Typowe przypadki błędów:

- niepoprawna flaga lub brak plików wejściowego/wyjściowego -> `ARGUMENTS_ERROR`,
- nieudane otwarcie pliku -> `FILE_ERROR`,
- błędny rekord w pliku grafu -> `INPUT_FORMAT_ERROR`,
- brak pamięci podczas tworzenia struktur pomocniczych -> `MEMORY_ERROR`.

9 Kompilacja i testowanie

9.1 Kompilacja

Projekt wykorzystuje `Makefile`:

- `make` - buduje aplikacje glowna do `bin/graph`,
- `make clean` - usuwa katalog `bin`,
- `make test` - buduje i uruchamia testy jednostkowe.

9.2 Testy

Zestaw testów obejmuje m.in. odczyt grafu, mapowanie identyfikatorów, działanie list sąsiedztwa oraz podstawowe przypadki dla obu algorytmów układania.